

DSA Problem Ladder — Day 2

Tuesday — Medium: decide what can be discarded

September 15, 2026 | Priyanka | 120 minutes | C11 or Python 3

Central question: what proves that skipping candidates is safe?

1. Merge reservations. Given unsorted closed intervals `[start, end]` with `start <= end`, return their union as sorted, non-overlapping intervals. Touching endpoints merge. Handle empty input, nesting, and equal starts. Require $O(n \log n)$ time.
2. Count pairs below a limit. Given an unsorted integer array and T , count index pairs `i < j` whose sum is strictly less than T . Equal values at different indices are different pairs. Require $O(n \log n)$ time; sorting a copy is allowed. Example: `[1, 1, 2, 3]`, $T=4$ gives 4. Use a wide count type in C.
3. Search after rotation. An array of distinct increasing integers was rotated an unknown number of positions. Return the index of a given target, or -1. Require $O(\log n)$ time and $O(1)$ space. Include empty, singleton, unrotated, and absent-target cases. Follow-up only: which inference fails if duplicates are allowed?

Session rules

- Minutes 0–10: closed-notes recall of one invariant and an edge case from the previous session (Monday: last week's boundary search).
- Minutes 10–110: solve today's queue in order. Check time after roughly 25 minutes per problem on easy days, 33 on medium days, and 50 on Wednesday. Unused time transfers forward; the queue is not a quota.
- Minutes 110–120: stop coding, record results, and explain one failed assumption and its correction. Mark unfinished work partial at the cutoff.
- Use C11 OR Python 3 consistently throughout the timed week. Translation is optional and does not count as another solve.
- Before coding, state a direct approach, its cost, and what property might remove work. After coding, give a correctness argument, time/space costs, and normal, boundary, and adversarial tests.
- After 15 minutes without a useful new step, write the blocker and choose to continue, request a small hint, or park the problem. Record hints; keep complete solutions closed during the attempt.

Count a solve only when working code meets the stated constraints, passes checks, and you can explain why it works. Separate independent and assisted solves. A correct slow baseline is partial if it misses the required bound. Testing time counts toward the session. If finished early, improve tests and proofs without increasing the solved count.

Attempt record

For each problem, submit its number, minutes spent, independent/assisted/partial/parked status, hints used, code, test results, invariant, and time/space analysis. Keep the same language throughout the timed week.

At 120 minutes record: attempted __; independently solved __; assisted solves __; partial __.

Exit ticket: Which assumption or boundary was hardest? Give one smallest failing example and explain your correction. Counts are evidence of this session, not a quota.